



ARQUITETURA DE SISTEMAS

AULA 13 - CLEAN ARCHITECTURE



INTRODUÇÃO

- ❑ Definição: Clean Architecture é um estilo de arquitetura de software proposto por Robert C. Martin (Uncle Bob) em 2012. Organiza o sistema em camadas concêntricas visando separar as regras de negócio dos detalhes de implementação.
- ❑ Influências: Baseia-se em ideias de arquiteturas anteriores (como Hexagonal - Ports & Adapters), todas focadas em divisão por camadas e forte separação de interesses (negócio vs. interfaces). Uncle Bob integrou esses conceitos em uma única abordagem prática.
- ❑ Objetivo Geral: Fornecer uma arquitetura independente de detalhes e focada no domínio. Isso resulta em sistemas mais fáceis de manter, testar e evoluir a longo prazo, mantendo o núcleo de negócio isolado de frameworks, banco de dados, UI e outros detalhes externos.

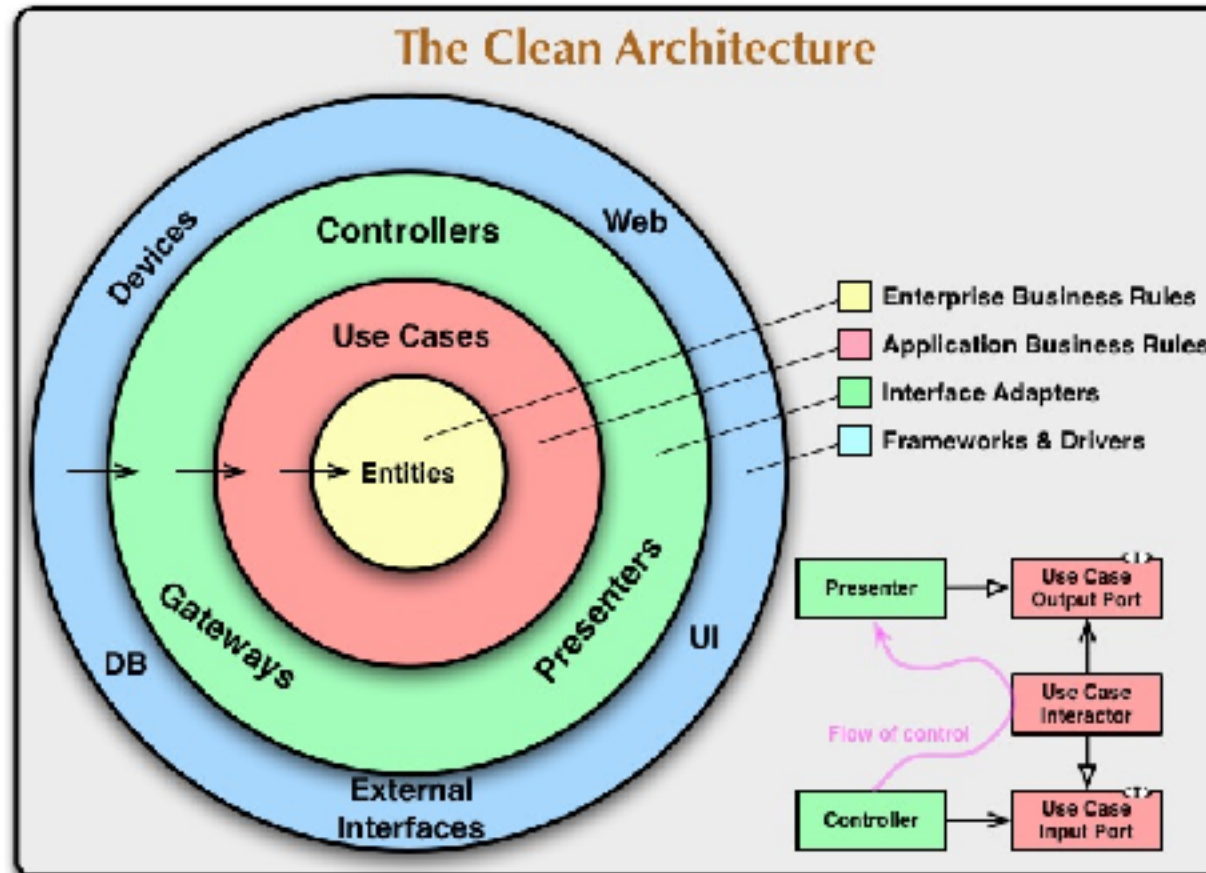


PRINCÍPIOS E BENEFÍCIOS FUNDAMENTAIS

- ❑ Independência de frameworks: A arquitetura não fica acoplada a nenhum framework ou biblioteca específico. Frameworks podem ser adotados como ferramentas plugáveis, em vez de o sistema ser “moldado” às restrições deles.
- ❑ Testabilidade: As regras de negócio podem ser testadas em isolamento, sem UI, banco de dados ou servidor web. Isso possibilita testes unitários dos casos de uso e entidades puras, garantindo verificação rápida da lógica sem depender de componentes externos.
- ❑ Independência da UI: A interface de usuário pode mudar sem impactar o restante do sistema. Por exemplo, uma aplicação web poderia ter sua UI substituída por um aplicativo mobile ou linha de comando, sem alterar as regras de negócio implementadas.
- ❑ Independência de banco de dados: O sistema não é ligado a um SGBD específico. Pode-se trocar um Oracle por MongoDB, SQL Server por PostgreSQL etc, sem mudanças nas regras de negócio. O domínio não “sabe” qual banco de dados está sendo usado.
- ❑ Independência de agentes externos: As regras de negócio não sabem nada sobre sistemas externos, dispositivos ou outros detalhes fora do núcleo. Isso inclui API de terceiros, detalhes de hardware, entre outros. Tais detalhes ficam nas camadas periféricas.



CAMADAS DA CLEAN ARCHITECTURE (VISÃO GERAL)



Fonte: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

Diagrama de referência da Clean Architecture, conforme proposta de Robert C. Martin. Os círculos internos (Entities e Use Cases) representam as políticas de negócio de mais alto nível, enquanto os círculos externos (Interface Adapters e Frameworks & Drivers) representam mecanismos de baixo nível. As setas ilustram o fluxo de controle típico entre camadas (por exemplo, um Controller chamando um caso de uso), mas as dependências de código devem sempre apontar para dentro, em direção às camadas de nível mais alto (Regra da Dependência).

CAMADAS DA CLEAN ARCHITECTURE (VISÃO GERAL)

- ❑ Entities: Camada mais interna. Contém as entidades de negócio do sistema, encapsulando regras corporativas gerais e dados essenciais do domínio.
- ❑ Use Cases: Contém as regras de aplicação ou casos de uso específicos. Orquestra as Entities para realizar funcionalidades que satisfazem os requisitos do usuário.
- ❑ Interface Adapters: Camada de adaptadores, responsável por converter dados entre o formato interno (domínio) e o formato externo (interfaces, banco, dispositivos).
- ❑ Frameworks & Drivers: Camada mais externa, inclui os frameworks, ferramentas e detalhes de infraestrutura (banco de dados, frameworks web, dispositivos externos, etc.) que suportam o sistema.



DEPENDENCY RULE

- ❑ Dependências apontam para dentro: Nenhum elemento em uma camada interna conhece ou depende de algo em camadas externas. Em outras palavras, qualquer nome (classe, função, variável) definido em uma camada externa não pode ser referenciado por código das camadas de nível mais alto. Essa regra garante que detalhes de implementação não “vazem” para o domínio.
- ❑ Políticas vs. detalhes: Quanto mais para o interior, mais alto o nível de abstração. As camadas internas definem políticas de negócio e conceitos abstratos, enquanto as camadas externas tratam dos detalhes concretos (por exemplo, APIs web, mecanismos de BD). Assim, o núcleo do sistema lida com regras, e as extremidades lidam com implementações.
- ❑ Dados desacoplados: Informações que cruzam os limites de camada devem ser passadas em formatos simples, compatíveis com o lado interno. Por exemplo, um registro de banco de dados ou um JSON da web deve ser transformado em um DTO ou objeto de domínio antes de chegar às regras de negócio. Não se deve passar um objeto dependente de framework (como um ResultSet JDBC) para dentro do caso de uso, pois isso violaria a Regra da Dependência.



INVERSÃO DE DEPENDÊNCIAS E DE CONTROLE

- ❑ Princípio da Inversão de Dependência (DIP): Clean Architecture aplica o DIP em nível arquitetural, garantindo que módulos de alto nível (regra de negócio) dependam apenas de abstrações, nunca de módulos de baixo nível. Na prática, isso significa usar interfaces para que a direção da dependência seja invertida a favor das camadas internas, respeitando a Regra da Dependências.
- ❑ Cruzar limites com interfaces: Quando uma camada interna precisa acionar uma operação de uma camada externa (ex: caso de uso salvar dados no BD), define-se uma interface no lado interno (um port de saída) que a camada externa implementará. Assim, o caso de uso chama um método da interface (sem conhecer a implementação concreta), e um adaptador externo cumpre a função real. Esse mecanismo de “porta e adaptador” utiliza polimorfismo para resolver o aparente contraditório fluxo de controle vs. dependência.
- ❑ IoC e injeção de dependência: Para fornecer as implementações concretas às interfaces do domínio, utilizamos Inversão de Controle (IoC). Por exemplo, um container de injeção de dependência ou o próprio código do main instanciam os adaptadores externos e os passam para o caso de uso. Dessa forma, o fluxo de execução acontece do controlador → caso de uso → interface → adaptador externo, mas as dependências de código continuam apontando do adaptador (externo) para a interface definida no domínio (interno).
Resultado: o núcleo chama abstrações, e a infraestrutura injeta os detalhes concretos conforme necessário.



ENTITIES

- ❑ Definição: Representam as regras de negócio de mais alto nível e abrangentes do sistema: as entidades de domínio fundamentais. Uma Entity pode ser implementada como um objeto com métodos ou como uma estrutura de dados com funções associadas; o importante é que encapsula conceitos e regras essenciais que poderiam existir independentemente de qualquer aplicação específica.
- ❑ Independência: Entidades são isoladas de detalhes externos. Alterações na interface, no banco de dados ou em frameworks não devem afetar as Entities. Elas são estáveis frente a mudanças operacionais. Por exemplo, uma entidade não muda se você trocar de Oracle para MySQL, ou de aplicação web para desktop, pois suas regras não dependem desses detalhes.
- ❑ Exemplo: Considere a entidade Empréstimo em um sistema financeiro. Ela reúne dados cruciais (valor do empréstimo, taxa de juros, prazo) e regras de negócio para cálculo de juros e validação de crédito. Essas regras (como “cobrar N% de juros ao mês”) fariam sentido mesmo fora do software, e portanto a entidade Empréstimo deve existir e operar sem depender de como o cálculo será exibido na UI ou armazenado no BD. Assim, mesmo se a interface do sistema mudar, a lógica de cálculo de empréstimo permanece centralizada e consistente.



USE CASES

- ❑ **Definição:** São as regras de negócio específicas da aplicação. Cada caso de uso implementa um serviço ou funcionalidade que o sistema oferece, orquestrando entidades para atingir um objetivo do usuário. Em termos simples, um Use Case realiza um fluxo de interação do sistema (por exemplo, “Cadastrar Cliente”, “Calcular Empréstimo”), definindo passo a passo como as Entities serão utilizadas para satisfazer uma ação do usuário.
- ❑ **Isolamento:** Os casos de uso dependem apenas das Entities e de estruturas de dados simples. Mudanças nos detalhes externos (como formato de tela, tipo de banco de dados) não afetam os casos de uso. Da mesma forma, alterações em regras internas do domínio (Entities) não devem exigir reescrever a lógica de fluxo do caso de uso, a menos que o próprio objetivo do caso de uso tenha mudado. Essa separação garante que podemos evoluir regras de negócio sem “quebrar” integrações externas, e vice-versa.
- ❑ **Exemplo:** O caso de uso “Criar Empréstimo” em um banco orquestra a validação dos dados do cliente, consulta sua pontuação de crédito e, se tudo estiver ok, cria uma nova entidade Empréstimo. Ele então solicita a persistência desse empréstimo através de uma interface de repositório, e retorna um resultado (por exemplo, aprovação ou rejeição). Note que toda essa lógica acontece na camada de aplicação: o caso de uso não sabe como a validação é exibida na tela (UI) nem como o empréstimo é salvo no banco (esses detalhes são delegados a outras camadas).



INTERFACE ADAPTERS

- ❑ **Papel:** São componentes que fazem a ponte entre o domínio e os agentes externos, convertendo dados e chamadas de um lado para o outro. Em outras palavras, adaptam o formato mais conveniente para as camadas internas (Entities/Use Cases) para o formato exigido por algo externo, como uma interface de usuário, banco de dados ou serviço de terceiros.
- ❑ **Exemplos de componentes:** Nesta camada se encaixam controladores MVC, presenters, views, gateways e repositórios. Por exemplo, um Controller web recebe uma requisição HTTP, extrai os dados (strings/JSON) e os converte em objetos de domínio ou DTOs, então invoca um Use Case. Ao retornar o resultado, o controller adapta a resposta de objeto de domínio para um formato HTTP/JSON de resposta. Outro exemplo: um Repository de banco de dados implementa uma interface do domínio (definida na camada de Use Case) e é responsável por converter objetos de domínio em comandos SQL, e resultados do banco (como linhas/colunas) de volta em objetos ou estruturas simples para o domínio.
- ❑ **Isolamento do domínio:** Todo o código específico de tecnologias externas fica nos adaptadores. Nenhum código nas camadas de Use Cases ou Entities acessa diretamente o banco de dados, a API externa ou detalhes de UI. Por exemplo, queries SQL e chamadas HTTP externas devem estar confinadas aos adaptadores de persistência ou gateways respectivos. Isso assegura que podemos mudar detalhes de implementação (uma API REST por gRPC, um banco SQL por NoSQL) modificando apenas os adaptadores, sem impactar as regras de negócio.



FRAMEWORKS & DRIVERS (CAMADA EXTERNA)

- ❑ Descrição: Camada mais externa, onde residem os frameworks, ferramentas e drivers que sustentam a aplicação. Aqui entram o servidor web, o banco de dados em si, frameworks de UI, sistema de arquivos, dispositivos externos, etc. Em geral, escreve-se pouco código nessa camada. Ela principalmente configura e inicializa os elementos do framework, repassando o controle para os adaptadores internos.
- ❑ Detalhes de Implementação: Tudo que é detalhe técnico vive nessa camada. O banco de dados é considerado um detalhe, assim como o web framework. Por isso, mantemos esses elementos no anel externo, “onde causam pouco dano” ao design de software. Por exemplo, é aqui que ficam arquivos de configuração (p.ex. application.properties do Spring Boot), inicialização do servidor HTTP, definição de rotas, instância concreta de um driver de BD, etc. Se amanhã decidirmos trocar o banco ou framework, é nesta camada que ocorrerá a mudança, enquanto o domínio permanece intacto.
- ❑ Código de integração (Glue Code): O código escrito nessa camada costuma apenas conectar os componentes. Exemplo: no Spring, a classe com método main que lança a aplicação pertence a essa camada; ela configura os beans do framework (controladores, serviços, repositórios) e inicia o servidor. Essa classe main conhece os controladores (adaptadores) e caso de uso principal a ser chamado no start, mas não contém lógica de negócio em si. Assim, Frameworks & Drivers atua como um encaixe entre o mundo externo e a arquitetura interna.



SEPARAÇÃO ENTRE DOMÍNIO E INFRAESTRUTURA

- ❑ Domínio vs. detalhes: Clean Architecture estabelece uma divisão clara entre o Domínio (camadas de Entities e Use Cases, que contêm a lógica de negócio) e a Infraestrutura (camadas de Interface Adapters e Frameworks & Drivers, que fornecem mecanismos técnicos). O código de domínio é independente das tecnologias externas, enquanto o código de infraestrutura existe unicamente para dar suporte ao domínio (conectando-o com UI, DB, etc.).
- ❑ Baixo acoplamento: Graças a essa separação, obtém-se baixo acoplamento. Pode-se alterar ou substituir detalhes de infraestrutura com impacto mínimo no domínio. Por exemplo, trocar o banco de dados ou modificar a interface gráfica requer mudanças apenas nas camadas externas: as regras de negócio permanecem intocadas. Inversamente, podemos alterar um processo de negócio (caso de uso ou regra em uma Entity) sem precisar reescrever código de interface ou de acesso a dados. Cada parte evolui em seu próprio ritmo.
- ❑ Longevidade e manutenibilidade: Essa arquitetura favorece a manutenção a longo prazo do sistema. Quando uma parte externa se torna obsoleta (um framework descontinuado, um DB ultrapassado), podemos substituí-la facilmente justamente porque o domínio está isolado e protegido dessas mudanças. Em suma, o núcleo de negócio (que representa o verdadeiro valor do software) mantém-se estável, enquanto detalhes de implementação podem ser adicionados, removidos ou atualizados sem efeitos colaterais profundos.



ARQUITETURAS FRAMEWORK-CENTRIC

- ❑ Sintomas: Uma arquitetura “centrada em framework” é aquela em que a estrutura do projeto é ditada pelo framework utilizado, e não pelas necessidades do domínio. Novos desenvolvedores abrem o projeto e veem pastas nomeadas pelos padrões do framework (controllers, services, repositories), mas têm dificuldade em identificar quais são as funcionalidades ou regras de negócio implementadas. Em outras palavras, o sistema grita o nome do framework em vez de gritar seu domínio de aplicação.
- ❑ Problemas: Quando a arquitetura é fornecida pelo framework, o código de negócio fica acoplado às convenções e restrições daquela ferramenta. Isso vai contra o princípio de arquitetura independente: se sua estrutura gira em torno de um framework, ela não está girando em torno dos seus use cases. As consequências são baixa flexibilidade (dificuldade para trocar o framework futuramente, já que afetaria todo o projeto) e teste mais complicado (se partes do negócio dependem diretamente do framework). Em suma, o framework passa a ditar decisões de design que deveriam ser do arquiteto de software.
- ❑ Abordagem Clean: Frameworks são ferramentas, não alicerces da arquitetura. Clean Architecture trata frameworks (sejam web, ORMs, etc.) como detalhes implementáveis periféricos. Idealmente, deve-se adiar decisões de escolha de framework até que a arquitetura de casos de uso esteja definida. Mesmo após escolhido, o framework fica isolado: por exemplo, usamos um controller Spring Boot como adaptador, mas o núcleo não depende das anotações ou classes do Spring. Isso permite trocar o framework ou atualizá-lo com impacto controlado, mantendo o foco da arquitetura nos casos de uso de negócio, não nas ferramentas.



ARQUITETURAS DATABASE-CENTRIC

- ❑ Sintomas: Projetos iniciados “pelo banco de dados”, onde o primeiro artefato é o esquema relacional, e o código de negócio é escrito em torno das tabelas. Nesses sistemas, é comum que entidades anêmicas (meros objetos espelho das tabelas) circulem por todas as camadas, e muita lógica de negócio acabe concentrada em stored procedures ou espalhada em consultas SQL nas camadas de persistência. A arquitetura acaba refletindo demais o modelo de dados, em vez de refletir os casos de uso.
- ❑ Problemas: Uma arquitetura centrada em BD tende a violar os princípios de independência. As camadas de negócio conhecem detalhes de SQL ou da estrutura do banco, criando alto acoplamento. Testar regras de negócio torna-se difícil sem um banco real (ou simulando conexões), o que aumenta a fragilidade. Além disso, evoluir o domínio exige mexer no esquema e em várias partes acopladas. Trocar de banco (por necessidade de escala ou custo) seria caro, pois as regras estão intrincadas com consultas e entidades do ORM. Em resumo, o BD deixa de ser um detalhe e passa a ser o eixo central: exatamente o contrário do preconizado pela Clean Architecture.
- ❑ Abordagem Clean: Tratar o banco de dados como um detalhe de infraestrutura, mantendo-o nos limites externos do sistema. Na prática, define-se no domínio interfaces de repositório com métodos expressando intenções (por ex: SalvarPedido), e implementa-se essas interfaces em adaptadores que conversam com o BD. Assim, o domínio não sabe se os dados vêm de um MySQL, um arquivo ou memória (ele apenas interage com abstrações). Consequentemente, trocar o banco de dados requer substituir apenas a implementação do repositório. Durante testes, pode-se usar uma implementação fake ou em memória dessas interfaces, permitindo testar a lógica de negócio sem um banco real.



TESTABILIDADE E QUALIDADE DO CÓDIGO

- ❑ Unidades testáveis: Seguindo Clean Architecture, as partes centrais do sistema (use cases, entidades) são naturalmente testáveis em isolamento. Podemos escrever testes unitários para um caso de uso complexo sem precisar levantar servidor web ou banco de dados (simulamos os repositórios e interfaces via mocks). Isso torna a bateria de testes mais rápida e confiável, permitindo identificar regressões de lógica de negócio facilmente.
- ❑ Sem dependência de UI/BD nos testes: Como o domínio não depende de frameworks, podemos testar regras de negócio sem envolver frameworks de UI ou detalhes de SQL. Por exemplo, para testar um cálculo financeiro de uma Entity, não é necessário nenhum browser ou conexão: apenas instanciamos a entidade em memória e chamamos seus métodos. Casos de uso podem ser testados fornecendo-se implementações fakes das interfaces de repositório, em vez de um BD real. Isso elimina efeitos colaterais e facilita até a prática de TDD, já que os desenvolvedores conseguem rodar testes a cada mudança sem configurações pesadas.
- ❑ Design mais limpo: A alta testabilidade é praticamente um indicador de bom design. Se um componente é difícil de testar isoladamente, geralmente é porque ele depende demais de contextos externos (UI, BD, global state). Ao aderir à Clean Architecture, obtemos classes e módulos focados e com responsabilidades claras, que por isso mesmo são simples de testar. O resultado é um código com baixo acoplamento e alta coesão, características que não apenas facilitam testes, mas também tornam a manutenção e evolução mais seguras (menos efeitos colaterais inesperados).



EVOLUÇÃO E MANUTENÇÃO A LONGO PRAZO

- ❑ Facilidade de mudança: Clean Architecture favorece a agilidade nas mudanças de software ao longo do tempo. Como as principais decisões de design isolam o núcleo de regras de negócio, podemos trocar detalhes de tecnologia com impacto mínimo. Por exemplo, se daqui a 5 anos surgir um framework web melhor, ou um banco mais escalável, podemos adotá-los alterando somente as camadas externas (os casos de uso e entidades permanecem funcionando). Assim, o sistema se adapta à evolução tecnológica sem precisar ser reescrito do zero.
- ❑ Atrasar decisões de tecnologia: Um princípio importante é adiar decisões de infraestrutura. Você não precisa decidir todos os frameworks e ferramentas no início do projeto: pode primeiro modelar o domínio e casos de uso. Frameworks, DBs, UI podem ser escolhidos e integrados quando necessário, e trocados depois se deixarem de atender. Isso mantém suas opções em aberto por mais tempo, evitando ficar preso a uma tecnologia que pode se tornar obsoleta.



EVOLUÇÃO E MANUTENÇÃO A LONGO PRAZO

- ❑ Foco no domínio (longevidade): O domínio de negócio tende a mudar menos que a tecnologia. Clean Architecture coloca o foco arquitetural no domínio, garantindo que essa parte mais estável do software seja facilmente compreendida e modificada. Quando um novo desenvolvedor olha o projeto, ele identifica rapidamente quais são as funcionalidades e regras implementadas: a estrutura do código sinaliza o propósito do sistema. Isso torna o onboarding de times mais rápido e reduz dependência de conhecimento tácito.
- ❑ Custo-benefício no longo prazo: Inicialmente, projetar com Clean Architecture pode adicionar um pouco de sobrecarga (mais classes/interfaces, necessidade de disciplina nas dependências). Porém, ao longo do ciclo de vida do software, essa inversão de controle paga dividendos: menos efeitos cascata ao realizar mudanças, menos “quebras” inesperadas, facilidade de teste e refatoração. Em sistemas de longa duração, a redução de custos de manutenção e de riscos de falhas supera em muito o esforço inicial, justificando a abordagem limpa.



CLEAN ARCHITECTURE E MICROSERVICES

- ❑ Escopo diferente, complementares: Microservices arquitetam um sistema como um conjunto de serviços pequenos e independentes (separação em nível de implantação). Já Clean Architecture organiza cada unidade de software internamente (separação em nível de código). Ambas se complementam: em uma arquitetura de microserviços, podemos (e devemos) aplicar Clean Architecture dentro de cada serviço. Isso assegura que cada microserviço tenha seu domínio interno bem isolado, facilitando manutenção individual de cada serviço.
- ❑ Boundary Contexts bem definidos: Microserviços geralmente podem seguir o Domain-Driven Design, onde cada serviço lida com um bounded context de domínio. Clean Architecture reforça essa abordagem, garantindo que dentro de cada serviço as camadas de domínio, caso de uso e infraestrutura estejam separadas. Assim, mesmo com vários serviços independentes, evitamos acoplamento interno em cada um: cada serviço pode evoluir seu domínio e detalhes de infraestrutura de forma autônoma, desde que mantenha contratos (APIs) estáveis com os demais.



CLEAN ARCHITECTURE E MICROSERVICES

- ❑ Tecnologia intercambiável: Uma das promessas dos microsserviços é permitir usar tecnologias diferentes em serviços diferentes. Clean Architecture leva isso adiante ao permitir tecnologias diferentes até dentro do mesmo serviço ao longo do tempo. Por exemplo, o time pode começar um microservice em Java com Spring e JPA; se no futuro for benéfico, poderiam trocar a parte de persistência para outra base de dados, ou migrar a API REST para GraphQL. E graças à arquitetura limpa, a maior parte do serviço (casos de uso e entidades) permaneceria igual, trocando-se apenas os adaptadores correspondentes. Cada serviço fica mais livre para adaptar-se tecnologicamente sem reescrever sua lógica de domínio.
- ❑ Monólitos modulares: Mesmo em aplicações monolíticas, aplicar Clean Architecture traz benefícios similares. Você pode estruturar o monólito em módulos internos seguindo as camadas limpas, obtendo baixo acoplamento interno. Isso inclusive facilita uma futura migração para microsserviços: se um dia decidir extrair um pedaço do monólito para um serviço separado, as fronteiras já estarão definidas (um conjunto de casos de uso e entidades bem encapsulado), tornando a extração muito mais simples e com risco reduzido.



CONCLUSÃO

- ❑ Arquitetura voltada ao domínio: Clean Architecture coloca o negócio no centro. A estrutura do projeto evidencia o que o sistema faz, em vez de detalhes de como ele faz. Isso significa que, ao ler o código, encontramos primeiro casos de uso, regras e entidades representando conceitos do domínio da aplicação. Os detalhes de frameworks, UI, BD aparecem apenas onde são necessários. Essa inversão de prioridades produz sistemas cujo design reflete melhor as necessidades de negócio, facilitando comunicação e entendimento.
- ❑ Preparada para o futuro: Ao obedecer a regras simples (dependências internas, separar camadas) conseguimos um sistema intrinsecamente flexível. Quando partes externas do sistema ficam obsoletas ou precisam evoluir, podemos atualizá-las com mínimo de impacto no todo. Isso prolonga a vida útil do software e protege o investimento feito no desenvolvimento inicial, já que o núcleo de valor (as regras de negócio) resiste às mudanças tecnológicas.



CONCLUSÃO

- ❑ Disciplina arquitetural: Clean Architecture exige disciplina por parte da equipe. Por exemplo, criação de portas e adaptadores, respeito às dependências, mais interfaces. Porém, essa estruturação clara atua como um guia durante todo o desenvolvimento. Em equipes experientes, é comum adotar esses princípios para grandes projetos, pois evita a degradação da arquitetura ao longo do tempo. Cada nova funcionalidade encontra seu lugar apropriado (novo caso de uso, talvez novos adaptadores) em vez de ser “jogada” em um lugar qualquer.
- ❑ Recapitulando: Diferentes dos estilos arquiteturais de alto nível (microservices, event-driven, etc.) vistos em aulas anteriores, Clean Architecture foca na arquitetura interna de cada componente ou sistema. Seus princípios (separação de camadas, inversão de dependências, etc) garantem que, independentemente de mudanças em UI, frameworks ou banco de dados, o core de negócio permaneça consistente e fácil de manter ao longo dos anos. Em última instância, é uma abordagem que busca sustentabilidade do software, facilitando tanto o trabalho do desenvolvedor no dia a dia quanto a evolução do produto no longo prazo.



ATÉ A PRÓXIMA!